

SAE 3 : Projet Domotique

Ly Minh Quan

Enseignant : M. GIBERT

Sommaire :

1. Introduction du projet	
2. Présentation des modules et fonctionnement.....	
3. Schéma fonctionnel & structurel.....	
3.1. Schéma structurel	
3.2. Schéma fonctionnel.....	
4. Explication du programme Arduino/ HTML/ JavaScript/ CSS	
4.1. Structure du code.....	
4.2. Boucle principale et sous-programmes.....	
4.3. Optimisation de code.....	
4.4. Détaille du programme Arduino (fonction par fonction).....	
4.5. Détaille du programme HTML & JavaScript(fonction par fonction).....	
4.6. Résultat sur page WEB.....	
5. Conclusion.....	
5.1. Bilan & Perspectives d'amélioration.....	
6. Annexes (Intégralité du code Arduino + commentaire simple).....	

1. Introduction du projet

Dans le cadre du projet SAE3.ESE.01-02, nous avons entrepris la conception et la validation d'un assistant domotique destiné à un habitat individuel, qu'il s'agisse d'un appartement ou d'une maison. Ce projet a pour objectif de faciliter le contrôle et la surveillance de divers paramètres domestiques.

Pour ce faire il faut développer un système capable de collecter des données analogiques, telles que la température et l'intensité lumineuse, ainsi que des informations TOR (tout ou rien) ce dispositif repose sur un module ESP32-C6, qui hébergera une page web accessible localement ou de n'importe où, codée en HTML, JS et CSS, pour afficher et piloter les informations en temps réel grâce à une interface connectée à un code Arduino.

2. Présentation des modules et fonctionnement

Dans ce projet, nous utilisons plusieurs capteurs. Ces différents capteurs sont :

- un microcontrôleur ESP32C6
- un écran TFT SPI avec une fonction « TOUCH » en plus
- LECTEUR RFID-RC522
- un BMP280
- un BH1750
- MH MQ SENSOR
- Relais 5v 30.22.7.005.0010 / Transistor BS170

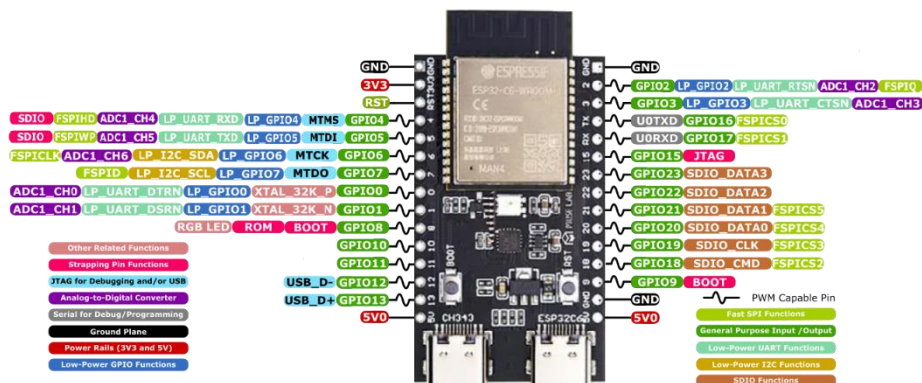


Figure 1 - Microcontrôleur ESP32

Le microcontrôleur ESP32C6, figure 1, est au centre de ce projet, offrant une utilisation puissante et simple pour gérer les différents capteurs et l'interface utilisateur. Il est doté d'une connectivité Wi-Fi intégrée et Bluetooth, ce qui permet à l'appareil de se connecter facilement à un réseau local pour la communication en ligne. L'ESP32C6 dispose également de plusieurs GPIO (entrées/sorties générales), d'une interface SPI pour la communication avec des composants externes comme l'écran TFT et le lecteur RFID, grâce à quoi, il permet une grande capacité de choix de connexion pour les capteurs que l'on utilise. S'ajoute à cela sa compatibilité avec l'IDE Arduino rend ainsi l'ESP32C6 adapté aux projets.



Figure 2 - Ecran TFT SPI + Touch

L'écran TFT SPI, figure 2, utilisé dans ce projet offre une interface visuelle pour l'utilisateur, permettant d'afficher des informations en temps réel issues des capteurs. Il utilise l'interface SPI pour une communication rapide et fiable avec l'ESP32C6, offrant des résolutions claires et nettes pour une meilleure lisibilité. De plus, l'écran est équipé d'une fonction « TOUCH » qui permet à l'utilisateur d'interagir directement avec l'écran, par exemple pour naviguer dans les menus ou appuyer sur des boutons pour déclencher des actions, comme la réinitialisation de certains paramètres.



Figure 3 – Lecteur RFID RC522

Le lecteur RFID RC522, figure 3, est un module qui permet de lire des tags RFID (Radio Frequency Identification). Dans ce projet, il est utilisé pour détecter des cartes ou des porte-clés RFID qui contiennent un identifiant unique. Le RC522 communique avec l'ESP32C6 via l'interface SPI et est capable de lire les informations distribuées sur les tags RFID à une fréquence élevée. Ce système est couramment utilisé dans des applications telles que la gestion d'accès, le contrôle de présence et l'authentification.

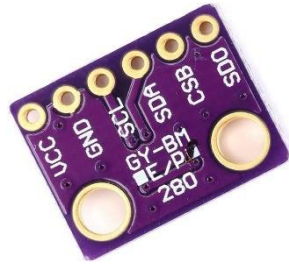


Figure 4 - Capteur BMP280

Le BMP280, figure 4, est un capteur de température et de pression barométrique de haute précision, utilisé dans ce projet pour mesurer la température ambiante et la pression atmosphérique. Ce capteur est idéal pour les applications intérieur voir même de surveillance de l'environnement. Le BMP280 utilise le protocole de communication I2C ou SPI pour transmettre les données à l'ESP32C6. Il est adapté aux projets qui nécessitent des mesures précises et fiables. En plus de la température et de la pression, il peut être utilisé pour estimer l'altitude et calculer la variation de pression.



Figure 5 - Capteur BH1750

Le BH1750, figure 5, est un capteur de lumière numérique conçu pour mesurer l'intensité lumineuse ambiante. Ce capteur fonctionne sur le principe de la photométrie, fournissant des lectures en lux (lx), une unité de mesure de l'intensité lumineuse. Le BH1750 communique avec l'ESP32C6 via le bus I2C, ce qui permet une intégration facile dans des projets de surveillance d'éclairage ou d'automatisation. Il est utilisé pour mesurer la luminosité dans l'environnement immédiat, ce qui peut être utile pour ajuster automatiquement l'éclairage dans des applications domotiques.



Figure 6 - Capteur MQ

Le capteur MQ (tel que le MQ-2, MQ-3, ou l'un des modèles similaire), figure 6, est utilisé pour détecter la concentration de différents gaz, notamment le monoxyde de carbone (CO), l'ammoniac (NH₃), le méthane (CH₄), et d'autres gaz organiques.

Dans ce projet, le capteur MQ est utilisé pour surveiller la qualité de l'air en mesurant la présence de gaz dans l'environnement. Il fonctionne en détectant les variations de résistance dans un matériau semi-conducteur, ce qui permet de calculer la concentration en gaz. Les données sont ensuite envoyées à l'ESP32C6 pour être traitées et affichées à l'écran. Ce capteur est particulièrement utile dans des applications de sécurité et de surveillance de la qualité de l'air, ainsi que dans des systèmes de détection de fuites de gaz. Il suffit tout simplement d'utiliser une formule pour avoir cette valeur en PPM (Partie Par Millions).



Figure 7 - Transistor BS170 et Relais Finder 30.22 de 5V

Ces 2 composants fonctionnent ensemble et ce relais, figure 7, est un interrupteur électromécanique qui, lorsqu'il est activé, peut contrôler des charges externes plus élevées, telles que des moteurs ou des lampes fonctionnant sous une tension supérieure (5 V). Dans ce projet, il est utilisé pour activer et désactiver des composants extérieurs via le signal du transistor.

Le relais fonctionne en recevant un signal du microcontrôleur via le transistor, ce qui permet un contrôle fiable et sans danger pour le microcontrôleur.

Ainsi, tous ces capteurs permettent d'avoir une carte complète et fonctionnelle en fonction programme utilisé.

3. Schéma structurel & fonctionnel

Pour représenter plus simplement le fonctionnement global de la carte électronique et du programme, un schéma structurel et fonctionnel est réalisé figure 8 & 9. Cela permet de comprendre comment les capteurs, le microcontrôleur, l'écran, et d'autres modules communiquent et travaillent ensemble pour fournir des données en temps réel.

3.1. Schéma structurel

Un schéma structurel, figure 8, permet de voir quel pin a été choisi pour chaque capteur et appareil sur la carte pour une compréhension du câblage des composants plus simple.

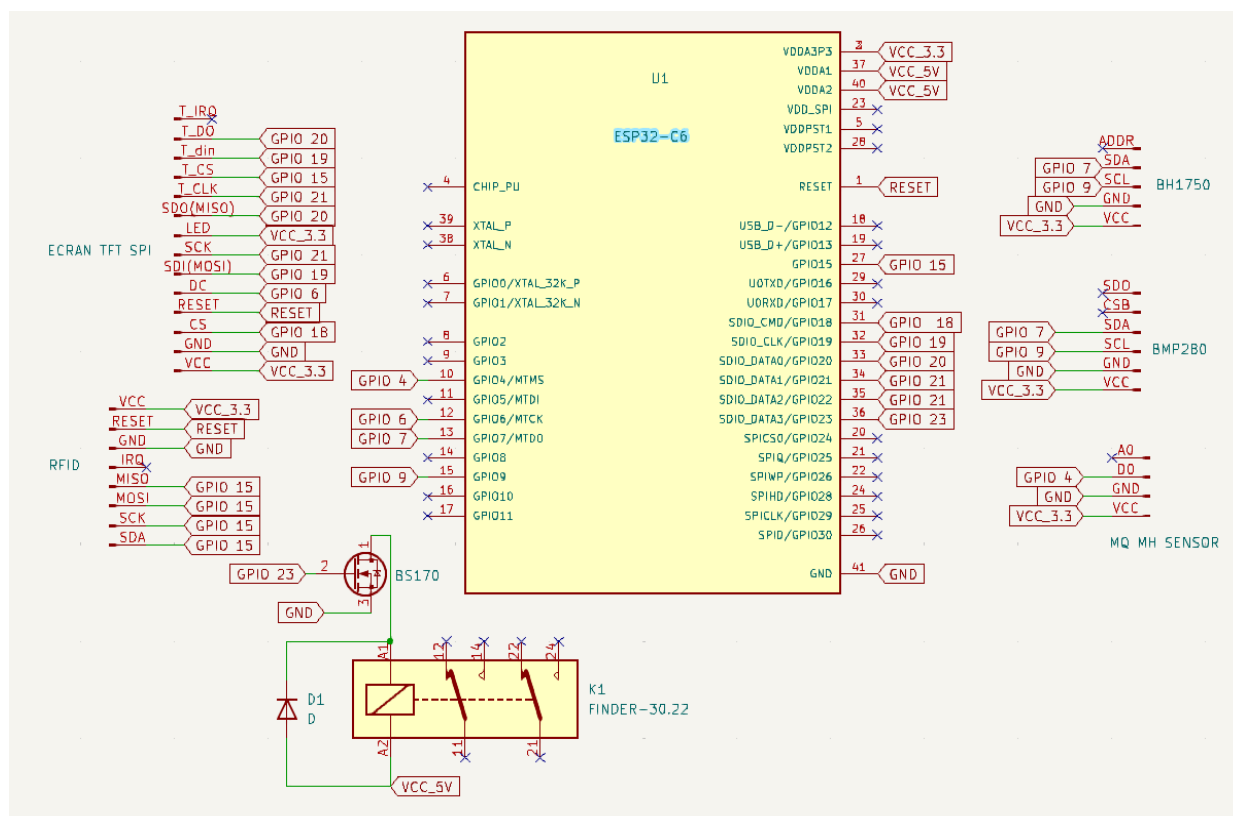


Figure 8 - Schéma Structurel de la carte domotique

3.2. Schéma fonctionnel

De manière à comprendre comment marche le programme fonctionne simple dans les grandes parties, il a fallu faire un schéma fonctionnel, figure 9.

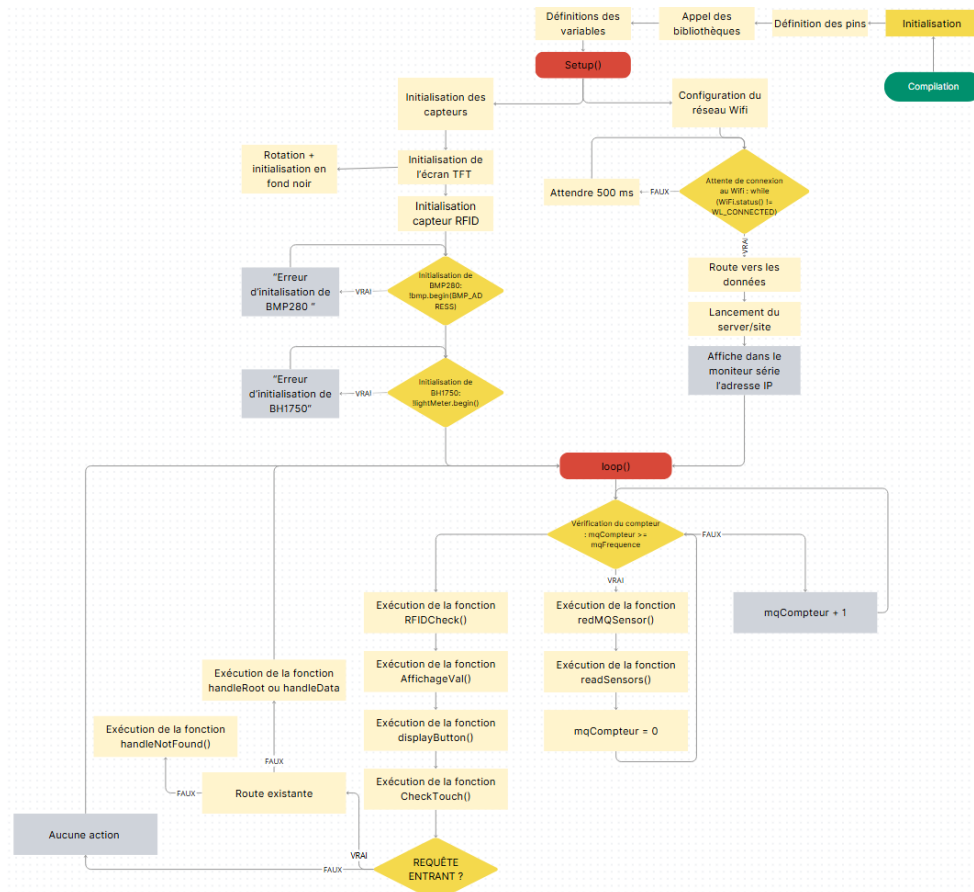


Figure 9 - Schéma Fonctionnel du programme

De ce schéma, figure 9, on en déduit qu'au début de la compilation du programme, s'exécute l'appel des bibliothèques, la définition des pins et variables utilisée dans tout le programme.

Ensuite les fonctions « setup() » et « loop() », dans le setup plusieurs actions sont faite tels que l'initialisation des capteurs (Écran TFT, lecteur RFID, BMP280...) et dans le cas des capteurs BMP280 et BH1750 en cas d'erreurs d'initialisation un message s'affiche dans le moniteur série.

En parallèle de cette partie, s'exécute aussi la configuration du réseau wifi et tant que la connexion wifi n'est pas établie, elle s'exécutera en boucle jusqu'à connexion au wifi (nom et mot de passe établi dans l'initialisation).

Ainsi dans le « loop() » toutes les fonctions nécessaire au fonctionnement du programme pour avoir le résultat voulu. Sois les fonctions: « RFIDCheck() », « AfficheVal() », « Button() », « blinkLED() » et « CheckTouch() » ainsi que « server.handleClient() » (= pour la partie WEB) et en parallèle à cela, j'ai ajouté à cela un compteur qui exécute les fonctions « redMQSensor() » et « readSensors() » lorsque « mqCompteur >= mqFrequence » est vrai.

Cette partie loop s'exécutera en boucle tout au long du programme.

4. Explication du programme Arduino/ HTML/ JavaScript/ CSS

4.1. Structure du code

La structure du code que j'ai été pensée pour augmenter la clarté et simplifiée la maintenabilité pour que chaque composant soit facilement identifiable et que les différentes sections soient bien séparées en fonction de leur fonction. L'inclusion des bibliothèques essentielles en début de fichier permet d'identifier rapidement les modules et logicielles utilisées dans le projet. Les constantes et définitions de broches sont déclarées à la suite des bibliothèques afin d'éviter la duplication et de faciliter des éventuelles modifications.

Les blocs fonctionnels sont organisés en sections dédiées, ce qui permet une compréhension aisée de la logique d'ensemble. Par exemple, le serveur web est configuré pour répondre aux requêtes HTTP, et les fonctions spécifiques aux capteurs ou modules sont regroupées par catégorie. Chaque capteur ou composant dispose de sa propre condition d'initialisation et de lecture, pour ainsi améliorer l'isolation des modules et simplifiant les tests individuels lors des protocoles de tests.

En structurant le code comme cela, il est également plus aisé d'ajouter de nouvelles fonctionnalités ou de réutiliser des segments dans d'autres projets en les commentant.

4.2. Boucle principale et sous-programmes

La boucle principale repose sur un schéma de contrôle non bloquant, grâce à l'utilisation de l'instruction « `millis()` », ce qui permet d'exécuter plusieurs tâches en parallèle sans interruption, comme le contrôle de la LED, la lecture des capteurs et la gestion des requêtes du serveur. En mettant en place des sous-programmes distincts pour chaque capteur et pour les interactions serveur, chaque fonction exécute une tâche unique bien définie, renforçant ainsi la lisibilité du code. Par exemple, la fonction « `readSensors()` » est dédiée uniquement à l'acquisition des valeurs des capteurs, tandis que « `RFIDCheck()` » s'occupe du contrôle des identifiants RFID. De plus, la fonction « `checkTouch()` » permet de créer une interaction tactile entre le bouton et l'utilisateur sans devoir gêner l'entièreté du programme.

Et ce genre de structure facilité grandement la maintenance et le débogage parce que chaque sous-programme peut être testé et/ou désactivé indépendamment du reste du programme au besoin juste en enlevant la ou les fonctions concerner, s'ajoute à cela le côté pratique, car s'il y a besoin de rajouter un nouveau composant ou module, il suffira tout simplement de rajouter la fonction qui lui est associé à la boucle « `loop()` », son initialisation au besoin dans le « `setup()` » ainsi que sa ou ces bibliothèques.

4.3. Optimisation de code

```
// Vérification du capteur à une fréquence plus lente
if (mqCompteur >= mqFrequence) {
    readMQSensor();
    readSensors();
    mqCompteur = 0;
}
mqCompteur++;
```

Figure 10 - Extrait de programme du compteur du setup()

Pour ce programme, figure 10, il a fallu faire en sorte de ne pas surcharger le microcontrôleur ESP32, donc pour ce faire, il ne faut pas faire trop de lecture à la suite auquel cas l'ESP32 ne suit plus et renvoie des valeurs lues sur les capteurs erronés.

Alors pour faire en sorte que cela n'arrive, j'ai dû y ajouter un compteur servant à limité sa fréquence d'appel soit en mettant les fonctions « readMQSensor() » et « readSensor() » dans le compteur, il est vrai que j'aurais pu mettre cela dans un « millis() » comme pour la fonction led.

```
// Lecture du capteur de fumée et calcule en PPM
void readMQSensor() {
    mqSensorValue = analogRead(MQ_SENSOR_PIN);
    float OutVoltage = (mqSensorValue / 1023.0) * Vcc;
    float Rs = RL * ((Vcc - OutVoltage) / OutVoltage);
    float ratio = Rs / R0;
    PPM = calculatePPM(ratio, -0.45, 2.3);

#ifdef Monit
    Serial.print("GAZ : ");
    Serial.println(PPM);
#endif
}

float calculatePPM(float Rs_Ro, float m, float b) {
    float logPPM = m * log10(Rs_Ro) + b;
    return pow(10, logPPM);
}
```

Figure 11 - Extrait de programme de la partie lecture du capteur de fumée

La fonction « calculatePPM() », figure 11, est utilisée pour calculer la présence de gaz en PPM grâce à une formule, et permettant ainsi de réduire le code redondant et accélèrent les ajustements lors de modifications des paramètres de calibration.

4.4. Détail du programme Arduino (fonction par fonction)

Dans le programme de l'ESP32, la fonction `setup()`, figure 12, s'exécute une seule fois au démarrage et prépare le microcontrôleur pour sa mission de surveillance et de communication. Elle initialise d'abord les périphériques (capteurs, relais, réseau Wi-Fi) et les différents composants nécessaires.

```
2024 // Configuration des Modules et de l'ESP32 et de la partie WEB
2025 void setup() {
2026     Serial.begin(115200);
2027     Wire.begin(7, 9);
2028
2029     pinMode(MQ_SENSOR_PIN, INPUT);
2030     pinMode(LED_PIN, OUTPUT);
2031     pinMode(RELAIS, INPUT);
2032
2033     tft.init();
2034     tft.setRotation(1);
2035     tft.fillScreen(TFT_BLACK);
2036     tft.setTouch(calData);
2037
2038     SPI.begin();
2039     rfid.PCD_Init();
2040
2041     if (!bmp.begin(BMP280_ADDRESS)) Serial.println("Erreur d'initialisation du BMP280 !");
2042     if (!lightMeter.begin()) Serial.println("Erreur d'initialisation du BH1750 !");
2043
2044     WiFi.begin(ssid, password);
2045     while (WiFi.status() != WL_CONNECTED) delay(500);
2046     if (MDNS.begin("esp32")) Serial.println("Répondeur MDNS démarré");
2047
2048     // Serial.print("Adresse IP de l'ESP32 : ");
2049     // Serial.println(WiFi.localIP());
2050
2051     server.on("/", handleRoot);
2052     server.on("/data", handleSensorData);
2053     server.on("/buttonState", handleButtonState);
2054     server.onNotFound(handleNotFound);
2055     server.begin();
2056     Serial.println("Serveur HTTP démarré");
2057 }
```

Figure 12 - Extrait de programme de la partie `setup()`

On y voit l'encadrer rouge, celui-ci correspond à l'initialisation des différents capteurs et modules tels que :

- « Wire.begin(7, 9) », définit les broches GPIO 7 et 9 comme les lignes SDA et SCL pour la communication avec les capteurs utilisant le protocole I2C.
- ligne 2033 à 2036, L'écran est initialisé et orienté pour faciliter l'affichage, puis une calibration de l'écran tactile est définie avec calData qui se trouve dans la création des variables.
- ligne 2038 et 2039, ces fonctions configurent la communication SPI pour le lecteur RFID.
- ainsi qu'une condition « if » (ligne 2041 & 2042) qui renvoie la message d'erreur dans le moniteur série lorsque l'un des capteurs BH1750 ou BMP280 ne s'initialise pas correctement dû à une mauvaise définition du « Wire.begin() ». Ce genre de message aide grandement pour vérifier d'où proviens le problème.

La boucle principale « loop() » est exécutée en continu. Elle regroupe les appels de fonctions pour le contrôle des capteurs, de l'interface, et des périphériques connectés.

Les fonctions exécutées en boucle sont :

```
2077 // Fonctions de Lecture des Capteurs
2078 void readSensors() {
2079     lightLevel = lightMeter.readLightLevel();
2080     altitude = bmp.readAltitude(1023);
2081     temperature = bmp.readTemperature();
2082     pressure = bmp.readPressure() / 100.0F;
2083
2084     if (altitude <= 0) altitude = 0.00;
2085
2086     #ifndef Monit
2087         Serial.print("Pression brute : ");
2088         Serial.println(altitude);
2089         Serial.print("Luminosité brute : ");
2090         Serial.println(lightLevel);
2091         Serial.print("Température brute : ");
2092         Serial.println(pressure);
2093         Serial.print("Température : ");
2094         Serial.println(temperature);
2095     #endif
2096 }
```

Figure 13 - Extrait du programme de la fonction de lecture des capteurs ("readSensors()")

- « readSensors() », figure 13, cette fonction lit les valeurs des capteurs de température, de pression, d'altitude et de luminosité avec un affichage sur le moniteur série de ses valeurs/données.

Pour avoir l'altitude, le capteur se sert de la pression ambiante, au lieu de mettre 1023, on aurait pu écrire directement « pressure » mais vu que cette valeur varie beaucoup et ne reste pas stable, il est difficile de l'utiliser même s'il affiche normalement environ 1023.

S'ajoute à ça une condition « if » qui définit à 0, l'altitude lorsque est négatif pour s'assurer de ne pas avoir des valeurs incompréhensibles.

```

2097 // Lecture du capteur de fumée et calcule en PPM
2098 void readMQSensor() {
2099     mqSensorValue = analogRead(MQ_SENSOR_PIN);
2100     float OutVoltage = (mqSensorValue / 1023.0) * Vcc;
2101     float Rs = RL * ((Vcc - OutVoltage) / OutVoltage);
2102     float ratio = Rs / R0;
2103     PPM = calculatePPM(ratio, -0.45, 2.3);
2104
2105     #ifdef Monit
2106         Serial.print("GAZ : ");
2107         Serial.println(PPM);
2108     #endif
2109 }
2110
2111 float calculatePPM(float Rs_Ro, float m, float b) {
2112     float logPPM = m * log10(Rs_Ro) + b;
2113     return pow(10, logPPM);
2114 }

```

Figure 14 - Extrait du programme de la fonction de lecture du capteur de fumée ("readMQSensor()")

- « readMQSensor() » et « calculatePPM() », figure 14, cette fonction lit la valeur du capteur de gaz (MQ) et puis la deuxième fonction convertit cette mesure en une estimation des PPM (parties par million) de gaz détecté pour cela, il utilise les valeurs -0.45 et 2.3 qui sont spécifiques aux caractéristiques du capteur et permettent d'obtenir une estimation des PPM grâce à une transformation logarithmique.

```

2128 // Fonctions d'Affichage sur l'Écran TFT
2129 void AffichageVal() {
2130     tft.setTextSize(2);
2131     tft.setTextColor(TFT_WHITE, TFT_BLACK);
2132
2133     tft.fillRect(20, 20, 200, 30, TFT_BLACK);
2134     tft.setCursor(20, 20);
2135     tft.printf("Temp: %.2f C\n", temperature);
2136
2137     tft.fillRect(20, 50, 200, 30, TFT_BLACK);
2138     tft.setCursor(20, 50);
2139     tft.printf("Press: %.2f hPa\n", pressure);
2140
2141     tft.fillRect(20, 80, 200, 30, TFT_BLACK);
2142     tft.setCursor(20, 80);
2143     tft.printf("Alt: %.2f m\n", altitude);
2144
2145     tft.fillRect(20, 110, 200, 30, TFT_BLACK);
2146     tft.setCursor(20, 110);
2147     tft.printf("Lum: %.2f lx\n", lightLevel);
2148
2149     tft.fillRect(20, 140, 200, 30, TFT_BLACK);
2150     tft.setCursor(20, 140);
2151     tft.printf("MQ: %.2f PPM\n", PPM);
2152
2153     tft.fillRect(20, 170, 200, 30, TFT_BLACK);
2154     tft.setCursor(20, 170);
2155     tft.printf("ID: %s", tagID.c_str());
2156 }

```

Figure 15 - Extrait du programme de la fonction d'affichage sur écran TFT ("AfficheVal()")

- « AffichageVal() », figure 15, cette fonction gère l'affichage des valeurs des capteurs sur l'écran TFT, avec une mise en forme et un positionnement fixe pour chaque valeur.

```

2116 // Contrôle du Module RFID
2117 void RFIDCheck() {
2118     if (rfid.PICC_IsNewCardPresent() && rfid.PICC_ReadCardSerial()) {
2119         tagID = "";
2120         for (byte i = 0; i < rfid.uid.size; i++) {
2121             tagID += String(rfid.uid.uidByte[i], HEX);
2122         }
2123         tagID.toUpperCase();
2124         rfid.PICC_HaltA();
2125     }
2126 }

```

Figure 16 - Extrait du programme de la fonction de lecture du lecteur RFID ("RFIDCheck()")

- « RFIDCheck() », figure 16, cette fonction vérifie la présence d'une carte RFID et enregistre son ID si une nouvelle carte est détectée. La fonction « rfid.PICC_ReadCardSerial() » lit le numéro unique de la carte détectée et le stocke sous forme de chaîne (dans « tagID »), ce qui permet de l'afficher ou de déclencher des actions selon l'ID.

```

2227 void Button() {
2228     // Change la couleur et le texte en fonction de l'état
2229     if (buttonPressed == false) {
2230         tft.fillRect(RESET_BTN_X, RESET_BTN_Y, RESET_BTN_WIDTH, RESET_BTN_HEIGHT, TFT_GREEN); // Couleur du bouton "APPUIE"
2231         tft.setCursor(RESET_BTN_X + 15, RESET_BTN_Y + 15); // Position du texte dans le bouton
2232         tft.setTextColor(TFT_BLACK, TFT_GREEN);
2233         tft.print("ON");
2234         digitalWrite(RELAIS, HIGH); // Allume le relais
2235     } else if (buttonPressed == true) {
2236         tft.fillRect(RESET_BTN_X, RESET_BTN_Y, RESET_BTN_WIDTH, RESET_BTN_HEIGHT, TFT_RED); // Couleur du bouton "NORMAL"
2237         tft.setCursor(RESET_BTN_X + 15, RESET_BTN_Y + 15);
2238         tft.setTextColor(TFT_BLACK, TFT_RED);
2239         tft.print("OFF");
2240         digitalWrite(RELAIS, LOW); // éteint le relais
2241     }
2242 }

```

Figure 17- Extrait de la fonction de contrôle du relais via le bouton

- La fonction « Button() », figure 17, gère l'affichage et le contrôle du relais en fonction de l'état du bouton. Le relais s'allume ou s'éteint en fonction de l'état du bouton, et l'affichage de l'écran TFT est mis à jour pour refléter cet état. Donc ce bouton n'est pas réel il juste présent sur l'écran TFT et le fait changer de couleur en fonction de son état.

```

2244 // Gestion du Touchscreen
2245 void checkTouch() {
2246     uint16_t touchX, touchY;
2247     if (tft.getTouch(&touchX, &touchY)) {
2248         if (touchX > RESET_BTN_X && touchX < RESET_BTN_X + RESET_BTN_WIDTH && touchY > RESET_BTN_Y && touchY < RESET_BTN_Y + RESET_BTN_HEIGHT) {
2249             // Alterne l'état du bouton
2250             buttonPressed = !buttonPressed;
2251             Button();
2252             // Mettre à jour le serveur Web avec l'état du bouton
2253             handleButtonState();
2254         }
2255     }
2256 }

```

Figure 18 - Extrait du programme de la fonction de détection tactile et basculement de l'état du bouton("checkTouch()")

- La fonction « Button() », figure 18, est réutiliser dans la fonction « checkTouch() » il détecte les interactions tactiles sur l'écran TFT et change l'état du bouton lorsqu'il est pressé. Lorsqu'un appui est détecté dans les limites de l'emplacement du bouton, l'état du relais et de l'affichage est modifié.

```

2217 // Fonctions d'État du LED
2218 void blinkLED() {
2219     unsigned long currentMillis = millis();
2220     if (currentMillis - previousMillis >= interval) {
2221         previousMillis = currentMillis;
2222         ledState = !ledState;
2223         digitalWrite(LED_PIN, ledState);
2224     }
2225 }

```

Figure 19 - Extrait du programme de la fonction de clignotement ("blinkLED()")

- « blinkLED() », figure 19, grâce à la fonction « millis() » cette fonction fait clignoter la LED intégrée de l'ESP32 à intervalles réguliers. Dans mon cas « interval », soit la variable qui choisit la fréquence de clignotement de ma led, elle clignote à une fréquence de 1 seconde par intervalle.

4.5. Détail du programme HTML & JavaScript(fonction par fonction)

Pour pouvoir avoir accès à une interface Web comme voulu dans le cahier des charges, il faut tout d'abord ajouter les options voulues dans le « setup() » dans le « loop() » ainsi que les fonctions qui vont avec et c'est ce qui a été fait.

Dans l'encadrer vert, figure 12, en plus des initialisations des capteurs et modules, il y est ajouté une partie web qui initialise et configure le serveur web pour permettre à l'ESP32 de répondre aux requêtes HTTP depuis un réseau local (ligne 2044 et 2045). Ensuite, il y est configuré le Multicast DNS qui permet d'accéder à l'ESP32 via un nom de domaine local sans connaître l'adresse IP exacte, ici "esp32.local". Donc la fonction « MDNS.begin("esp32") » est lancée le service mDNS avec le nom esp32.

J'y ajoute un affichage, en commentaire, de l'IP juste pour en cas de besoin faire des tests ou vérifier si cela marche bien.

Et en plus pour aller chercher les données des capteurs et les transmettre, j'y ai configuré des routes http pour que chaque « server.on() » définit une route pour une URL spécifique grâce à des fonctions tel que :

« / » : redirige vers la page principale via la fonction « handleRoot() ».

« /data » : réponds avec les données JSON des capteurs via « handleSensorData() ».

« /buttonState » : fournis l'état actuel du bouton par « handleButtonState() ».

« server.onNotFound(handleNotFound) » : déclenche la fonction « handleNotFound() » si une route non définie est demandée, renvoyant une page 404.

Et la configuration se termine avec le lancement du serveur http avec un message dans le moniteur série.

Pour exécuter toutes ces routes, on a besoin de fonctions qui le font. Donc, ici, les fonctions utilisées pour créer ces routes sont :

```

2160 void handleSensorData() {
2161     String json = "{";
2162     json += "\"temperature\":" + String(temperature) + ",";
2163     json += "\"pressure\":" + String(pressure) + ",";
2164     json += "\"altitude\":" + String(altitude) + ",";
2165     json += "\"lightLevel\":" + String(lightLevel) + ",";
2166     json += "\"PPM\":" + String(PPM) + ",";
2167     json += "\"rfidID\":" + tagID + "\"";
2168     json += "}";
2169     server.send(200, "application/json", json);
2170 }

```

Figure 20 - Extrait du programme de la fonction de création et envoi des données de capteurs au format JSON ("handleSensorData()")

- « handleSensorData() », figure 20, cette fonction génère une réponse JSON contenant les valeurs actuelles des capteurs et l'ID RFID pour que l'utilisateur puisse accéder aux données sous format structuré. Le JSON servira donc à stocker et à envoyer les données des capteurs. Et l'envoi à l'utilisateur se fera par le biais de la dernière ligne « server.send() ».

```

void handleButtonState() {
    server.send(200, "text/plain", buttonPressed ? "OFF" : "ON"); // Simplification du if / else
}

```

Figure 21 - Extrait du programme de la fonction de renvoi du bouton à la page WEB ("handleDuttonState()")

```

2214 if (digitalRead(RELAIS) == HIGH) server.send(200, "text/plain", "ON");
2215 else server.send(200, "text/plain", "OFF");
2216 |

```

Figure 22 - Simplification de la fonction figure 21

- « handleButtonState() », figure 21 & 22, cette fonction peut s'écrire plus simple sur figure [...], la fonction de cette fonction est de renvoyer « ON » ou « OFF » directement sur la page web, ainsi lorsqu'il est dans une position ou une autre, il activera ou non le relais qui y est connecté par le biais du transistor.

```

2212 // Gestion des Pages 404
2213 void handleNotFound() {
2214     server.send(404, "text/plain", "404: Page introuvable");
2215 }

```

Figure 23 - Extrait du programme de la fonction des requêtes inexistantes et renvoi d'erreur ("handlNotFound()")

- « handleNotFound() », figure 23, cette fonction gère les requêtes à des routes non-existantes, en renvoyant une page d'erreur 404 lorsqu'il ne trouve pas la route demander.


```

2172 void handleRoot() {
2173     // Partie HTML
2174     String html = "<!DOCTYPE html><html><head>";
2175     html += "<meta charset='UTF-8'>";
2176     html += "<title>Données des Capteurs</title>";
2177     html += "<style>body { font-family: Arial, sans-serif; }</style>";
2178     html += "</head><body>";
2179     html += "<h1>Données des Capteurs</h1>";
2180     html += "<p>Température: <span id='temperature'>0</span> °C</p>";
2181     html += "<p>Pression: <span id='pressure'>0</span> hPa</p>";
2182     html += "<p>Altitude: <span id='altitude'>0</span> m</p>";
2183     html += "<p>Luminosité: <span id='lightLevel'>0</span> lx</p>";
2184     html += "<p>Gaz (PPM): <span id='PPM'>0</span> ppm</p>";
2185     html += "<p>ID RFID: <span id='rfidID'></span></p>";
2186     html += "<p>État du bouton: <span id='buttonState'></span></p>";
2187
2188     // Partie JS
2189     html += "<script>";
2190     html += "setInterval(() => {";
2191     html += "fetch('/data').then(response => response.json()).then(data => {";
2192     html += "document.getElementById('temperature').innerText = data.temperature;";
2193     html += "document.getElementById('pressure').innerText = data.pressure;";
2194     html += "document.getElementById('altitude').innerText = data.altitude;";
2195     html += "document.getElementById('lightLevel').innerText = data.lightLevel;";
2196     html += "document.getElementById('PPM').innerText = data.PPM;";
2197     html += "document.getElementById('rfidID').innerText = data.rfidID;";
2198     html += "});";
2199     html += "fetch('/buttonState').then(response => response.text()).then(state => {";
2200     html += "document.getElementById('buttonState').innerText = state;";
2201     html += "});";
2202     html += "}, 1000);";
2203     html += "</script>";
2204     html += "</body></html>";
2205     server.send(200, "text/html", html);
2206 }

```

Figure 24 - Extrait du programme de la fonction affichage de la page Web ("handleRoot()")

- « handleRoot() », figure 24, c'est la fonction principale qui va générer la page HTML et qui affichera les données des capteurs, dans cette fonction il y a :
 - « String html = "<!DOCTYPE html><html><head>"; », cette ligne démarre la création d'un document HTML. HTML est une chaîne de caractères qui contiendra le code HTML complet envoyé au client. Donc chaque début de ligne de code HTML débutera avec « String html = " ... " ».
 - Pour donner un titre à la page, c'est la ligne 2176 et dans la ligne 2175, on y retrouve UTF-8 qui sert à définir l'encodage pour supporter tous les caractères et le titre de la page qui apparaîtra dans l'onglet du navigateur. Pour qu'il n'y ait pas de caractère incompris par le navigateur.
 - Pour avoir une certaine police d'écriture, il est nécessaire d'y ajouter un style CSS. Dans ce cas précis je n'ai que cette vue que ma page est très basique, il n'y a pas besoin de modifier l'apparence de ma page Web. Mais plus il y en a, plus le site sera beau et attractif pour les yeux.
 - Chaque paragraphe est associé à une valeur de capteur avec un <p> pour chaque donnée, et ces éléments seront mis à jour, à la moindre variation des valeurs des capteurs, par JavaScript avec les valeurs lues du serveur.
 - À la suite de la partie HTML s'en suit la partie JavaScript, cette partie JavaScript mettra à jour les valeurs de capteurs à intervalles réguliers.
 - Ligne 2190, cette fonction « setInterval() », c'est elle qui mettra à jour les données des capteurs sur la page et ceux à intervalle régulier.

- Pour rafraîchir la page avec de nouvelles données, il faut d'abord les récupérer, alors la fonction « fetch() » envoie une requête GET à la route /data, attend la réponse JSON et traite les données retournées via data
- À la ligne 2199 à 2201, c'est cette fonction qui va faire une requête à la route « /buttonState » et récupère l'état actuel du bouton pour mettre à jour l'élément buttonState avec "ON" ou "OFF".
- Et enfin les 3 dernières ligne avec « html += », clôture le JavaScript et le HTML. Ainsi la fonction « setInterval() » répètera le tout toutes les secondes.

4.6. Résultat sur page WEB

Données des Capteurs

Température: 25.97 °C

Pression: 1024.42 hPa

Altitude: 0 m

Luminosité: 0.83 lx

Gaz (PPM): 88.72 ppm

ID RFID:

État du bouton: RELAIS_OFF

Figure 25 - Page Web (affichage final)

On observe la page web finale de mon projet, il est possible de voir tout les capteurs et leurs données, données qui sont relativement cohérentes.

Il est possible de voir aussi que la page web est très simple sans couleur, il serait mieux d'y ajouter quelque chose qui rende la page plus attrayante.

5. Conclusion

5.1. Bilan & Perspectives d'amélioration

Dans ce projet de domotique, le but est d'offrir à un habitat (appartement ou maison) une interface de visualisation d'informations analogique (température, intensité lumineuse, gaz polluant...) mais aussi une interface de commande de sortie TOR permettant de piloter des relais (gâche électrique porte, Volet roulant, ouverture portail...). Tout ça est visible et pilotable via une page WEB codée sous HTML(JS, CSS) et interagir avec du code Arduino pour les données.

Ainsi, le programme utilisé est optimisé de façon qu'il assure une gestion simple et fluide de l'affichage et des contrôles. Avec ce programme, il sera aisé de faire des maintenance dessus vu que l'ESP32 a été configurée pour permettre de voir en temps réel ce que les différents modules et composants reçoivent comme données.

Mais ce programme pourrait être bien plus amélioré, comme par exemple, l'interface WEB pourrait être bien plus améliorée, au lieu de lire les valeurs, elles pourraient être représentées par une jauge dans le cas de la température et l'affichage pourrait inclure plus de CSS pour que la page WEB soit visuellement plus attrayante pour améliorer l'expérience de l'utilisateur ou encore ajouter plus de capteurs et modules pour le rendre encore plus performant.

Dans l'ensemble, ce projet a permis de démontrer la faisabilité d'un système domotique accessible et flexible, avec un potentiel d'extension important pour répondre à divers besoins domotiques à l'avenir.

6. Annexes (Intégralité du code Arduino + commentaire simple)

- Intégralité du programme Arduino et HTML :

```
// Bibliothèques et Définitions
#include <WiFi.h>
#include <NetworkClient.h>
#include <WebServer.h>
#include <ESPmDNS.h>
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_BMP280.h>
#include <BH1750.h>
#include <MFRC522.h>
#include <TFT_eSPI.h>
#include <TouchScreen.h>

// #define Monit

// Configuration Réseau à ce connecter
const char *ssid = "Freebox-78C617";
const char *password = "vh562qbszkqvz65zqk45wr";
WebServer server(80);

// Déclarations et constantes des capteurs
float temperature = 0.0;
float pressure = 0.0;
float altitude = 0.0;
float lightLevel = 0.0;
float PPM = 0.0;

const float Vcc = 5.0;
const float RL = 10.0;
const float R0 = 10.0;
int mqCompteur = 0;
const int mqFrequence = 15;

// Définition des Pins et Paramètres pour LED et autres composants
#define LED_PIN LED_BUILTIN
#define RESET_BTN_X 200
#define RESET_BTN_Y 180
#define RESET_BTN_WIDTH 90
#define RESET_BTN_HEIGHT 50

// Déclarations des variables pour la LED et la boucle Millis
unsigned long previousMillis = 0;
const long interval = 1000;
```

```

bool isStarted = false;
bool buttonPressed = true;
bool ledState = LOW;

// définition des Pins de l'écran TFT
#define MOSI 19
#define MISO 20
#define SCLK 21
#define SS 22
#define RST_PIN -1
#define TFT_CLK 5
#define TFT_CS 18
#define TFT_MOSI 19
#define TFT_MISO 20
#define TFT_SCLK 21
#define TFT_DC 6
#define TFT_RST -1
#define TOUCH_CS 15 // définitions du CS du TOUCH
uint16_t calData[5] = { 392, 3401, 381, 3306, 1 };

#define LED1 10
#define RELAIS 23
#define MQ_SENSOR_PIN 4
#define BMP280_ADDRESS 0x76

// Initialisation des Objets de Capteur
Adafruit_BMP280 bmp;
BH1750 lightMeter;
MFRC522 rfid(SS, RST_PIN);
TFT_eSPI tft = TFT_eSPI();

int mqSensorValue;
String cardID = "E378CFA9";
String tagID = "";

// Configuration des Modules et de l'ESP32 et de la partie WEB
void setup() {
  Serial.begin(115200);
  Wire.begin(7, 9);

  pinMode(MQ_SENSOR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  pinMode(RELAIS, INPUT);

  tft.init();
  tft.setRotation(1);
  tft.fillScreen(TFT_BLACK);

```

```

tft.setTouch(calData);

SPI.begin();
rfid.PCD_Init();

if (!bmp.begin(BMP280_ADDRESS)) Serial.println("Erreur d'initialisation du BMP280 !");
if (!lightMeter.begin()) Serial.println("Erreur d'initialisation du BH1750 !");

WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) delay(500);
if (MDNS.begin("esp32")) Serial.println("Répondeur MDNS démarré");

// Serial.print("Adresse IP de l'ESP32 : ");
// Serial.println(WiFi.localIP());

server.on("/", handleRoot);
server.on("/data", handleSensorData);
server.on("/buttonState", handleButtonState);
server.onNotFound(handleNotFound);
server.begin();
Serial.println("Serveur HTTP démarré");
}

// Boucle Principale
void loop() {
  blinkLED();
  RFIDCheck();
  AffichageVal();
  checkTouch();
  Button();

  // Vérification du capteur à une fréquence plus lente
  if (mqCompteur >= mqFrequence) {
    readMQSensor();
    readSensors();
    mqCompteur = 0;
  }
  mqCompteur++;
  server.handleClient();
}

// Fonctions de Lecture des Capteurs
void readSensors() {
  lightLevel = lightMeter.readLightLevel();
  altitude = bmp.readAltitude(1023);
  temperature = bmp.readTemperature();
  pressure = bmp.readPressure() / 100.0F;
}

```

```

if (altitude <= 0) altitude = 0.00;

#ifdef Monit
  Serial.print("Pression brute : ");
  Serial.println(altitude);
  Serial.print("Luminosité brute : ");
  Serial.println(lightLevel);
  Serial.print("Température brute : ");
  Serial.println(pressure);
  Serial.print("Température : ");
  Serial.println(temperature);
#endif
}

// Lecture du capteur de fumée et calcule en PPM
void readMQSensor() {
  mqSensorValue = analogRead(MQ_SENSOR_PIN);
  float OutVoltage = (mqSensorValue / 1023.0) * Vcc;
  float Rs = RL * ((Vcc - OutVoltage) / OutVoltage);
  float ratio = Rs / R0;
  PPM = calculatePPM(ratio, -0.45, 2.3);

#ifdef Monit
  Serial.print("GAZ : ");
  Serial.println(PPM);
#endif
}

float calculatePPM(float Rs_Ro, float m, float b) {
  float logPPM = m * log10(Rs_Ro) + b;
  return pow(10, logPPM);
}

// Contrôle du Module RFID
void RFIDCheck() {
  if (rfid.PICC_IsNewCardPresent() && rfid.PICC_ReadCardSerial()) {
    tagID = "";
    for (byte i = 0; i < rfid.uid.size; i++) {
      tagID += String(rfid.uid.uidByte[i], HEX);
    }
    tagID.toUpperCase();
    rfid.PICC_HaltA();
  }
}

// Fonctions d'Affichage sur l'Écran TFT
void AffichageVal() {

```

```

tft.setTextSize(2);
tft.setTextColor(TFT_WHITE, TFT_BLACK);

tft.fillRect(20, 20, 200, 30, TFT_BLACK);
tft.setCursor(20, 20);
tft.printf("Temp: %.2f C\n", temperature); // TEMPERATURE

tft.fillRect(20, 50, 200, 30, TFT_BLACK);
tft.setCursor(20, 50);
tft.printf("Press: %.2f hPa\n", pressure); // PRESSION

tft.fillRect(20, 80, 200, 30, TFT_BLACK);
tft.setCursor(20, 80);
tft.printf("Alt: %.2f m\n", altitude);

tft.fillRect(20, 110, 200, 30, TFT_BLACK);
tft.setCursor(20, 110);
tft.printf("Lum: %.2f lx\n", lightLevel); // LUMINOSITE AMBIANTE

tft.fillRect(20, 140, 200, 30, TFT_BLACK);
tft.setCursor(20, 140);
tft.printf("MQ: %.2f PPM\n", PPM); // FUMEE

tft.fillRect(20, 170, 200, 30, TFT_BLACK);
tft.setCursor(20, 170);
tft.printf("ID: %s", tagID.c_str()); // RFID
}

// Fonctions d'État du LED
void blinkLED() {
    unsigned long currentMillis = millis();
    if (currentMillis - previousMillis >= interval) {
        previousMillis = currentMillis;
        ledState = !ledState;
        digitalWrite(LED_PIN, ledState);
    }
}

void Button() {
    // Change la couleur et le texte en fonction de l'état ET SON AFFICHAGE
    if (buttonPressed == false) {
        tft.fillRect(RESET_BTN_X, RESET_BTN_Y, RESET_BTN_WIDTH, RESET_BTN_HEIGHT,
TFT_GREEN); // Couleur du bouton "ON"
        tft.setCursor(RESET_BTN_X + 15, RESET_BTN_Y + 15); // Position du
texte dans le bouton
        tft.setTextColor(TFT_BLACK, TFT_GREEN);
        tft.print("ON");
    }
}

```



```

    digitalWrite(RELAIS, HIGH); // Allume le relais
} else if (buttonPressed == true) {
    tft.fillRect(RESET_BTN_X, RESET_BTN_Y, RESET_BTN_WIDTH, RESET_BTN_HEIGHT,
TFT_RED); // Couleur du bouton "OFF"
    tft.setCursor(RESET_BTN_X + 15, RESET_BTN_Y + 15);
    tft.setTextColor(TFT_BLACK, TFT_RED);
    tft.print("OFF");
    digitalWrite(RELAIS, LOW); // éteind le relais
}
}

// Gestion du Touchscreen
void checkTouch() {
    uint16_t touchX, touchY;
    if (tft.getTouch(&touchX, &touchY)) {
        if (touchX > RESET_BTN_X && touchX < RESET_BTN_X + RESET_BTN_WIDTH && touchY
> RESET_BTN_Y && touchY < RESET_BTN_Y + RESET_BTN_HEIGHT) {
            // Alterne l'état du bouton
            buttonPressed = !buttonPressed;
            Button();
            // Mettre à jour le serveur Web avec l'état du bouton
            handleButtonState();
        }
    }
}

// PARTIE FONCTION HTML, JAVASCRIPT, CSS
// Gestion du Serveur Web et Affichage de Données
void handleSensorData() {
    String json = "{}";
    json += "\"temperature\":" + String(temperature) + ",";
    json += "\"pressure\":" + String(pressure) + ",";
    json += "\"altitude\":" + String(altitude) + ",";
    json += "\"lightLevel\":" + String(lightLevel) + ",";
    json += "\"PPM\":" + String(PPM) + ",";
    json += "\"rfidID\":" + tagID + "\"";
    json += "}";
    server.send(200, "application/json", json);
}

void handleRoot() {
    // Partie HTML
    String html = "<!DOCTYPE html><html><head>";
    html += "<meta charset='UTF-8'>";
    html += "<title>Données des Capteurs</title>";
    html += "<style>body { font-family: Arial, sans-serif; }</style>";
    html += "</head><body>";
    html += "<h1>Données des Capteurs</h1>";
}

```

```

html += "<p>Température: <span id='temperature'>0</span> °C</p>";
html += "<p>Pression: <span id='pressure'>0</span> hPa</p>";
html += "<p>Altitude: <span id='altitude'>0</span> m</p>";
html += "<p>Luminosité: <span id='lightLevel'>0</span> lx</p>";
html += "<p>Gaz (PPM): <span id='PPM'>0</span> ppm</p>";
html += "<p>ID RFID: <span id='rfidID'></span></p>";
html += "<p>État du bouton: <span id='buttonState'></span></p>";

```

```
// Partie JS
```

```

html += "<script>";
html += "setInterval(() => {";
html += "fetch('/data').then(response => response.json()).then(data => {";
html += "document.getElementById('temperature').innerText = data.temperature;";
html += "document.getElementById('pressure').innerText = data.pressure;";
html += "document.getElementById('altitude').innerText = data.altitude;";
html += "document.getElementById('lightLevel').innerText = data.lightLevel;";
html += "document.getElementById('PPM').innerText = data.PPM;";
html += "document.getElementById('rfidID').innerText = data.rfidID;";
html += "});";
html += "fetch('/buttonState').then(response => response.text()).then(state => {";
html += "document.getElementById('buttonState').innerText = state;";
html += "});";
html += "}, 1000);";
html += "</script>";
html += "</body></html>";
server.send(200, "text/html", html);
}

```

```

void handleButtonState() {
    server.send(200, "text/plain", buttonPressed ? "OFF" : "ON"); // Simplification du if /
    esle
}

```

```
// Gestion des Pages 404
```

```

void handleNotFound() {
    server.send(404, "text/plain", "404: Page introuvable");
}

```